

InsightAI: Root Cause Analysis in Large Log Files with Private Data Using Large Language Model

1st Maryam Ekhlasi 2nd Anurag Prakash 3rd Maxime Lamothe 4rd Michel Dagenais
Computer Software Department Ciena Computer Software Department Computer Software Department
Polytechnique Montreal Ottawa, Canada. Polytechnique Montreal Polytechnique Montreal
 Montreal, Canada. Montreal, Canada. Montreal, Canada.

maryam.ekhlasi@polymtl.ca

Abstract—[Problem] As industries increasingly depend on complex software systems, efficient log analysis is essential for maintaining reliability and privacy. However, identifying problems through logs is often time-consuming and costly for developers. [Background] Large language models (LLMs) can automate parts of log analysis, but challenges like limited computational resources and the frequent need to retrain LLMs due to the dynamic nature of software logs persist. External LLMs, such as GPTs, along with in-context learning techniques, can help reduce some of these issues, but other challenges, including token limitations, high token costs, and data privacy, remain. [Method] To tackle these challenges, we developed an automated pipeline that extracts log files and employs in-context learning, allowing the model to efficiently adapt to changes without extensive retraining. Our approach introduces a novel flame-graph-like method that reduces token usage, thereby lowering token-related costs and response latency while maintaining high accuracy. [Results] This solution allows industries to automate log analysis, minimize system downtime, and enhance performance, all while keeping data privacy and maintaining operational efficiency. [Conclusion] Our flame-graph-like methodology reduces input tokens by 93.61% and processing latency by 77.45%. Our anonymization results show an improvement of 138.63% over the baseline. This industrial experience report presents our approach to allow industries to balance token costs, maintain response accuracy, and ensure data privacy while relying on external LLMs without the need to manage computational resources directly.

Index Terms—LLM, ChatBot, Software Logs, Token Limitation, Data Privacy, In-Context Learning, Prompt Engineering.

I. INTRODUCTION

The complexity and size of modern software systems produce massive amounts of logs. logging is now crucial practice for maintaining software reliability [1]–[3]. They can reveal various issues during production, such as hardware failures, misconfigurations, poor coordination between services [4]. They can be also used for anomaly detection [5]–[8], failure prediction [9], [10], and root cause analysis [11], [12]. Since these logs can grow to gigabytes in size, manually reviewing them becomes impractical. Automating log analysis is therefore essential for efficiently identifying and addressing problems [13].

To address these challenges, Large Language Models (LLMs) have demonstrated considerable potential for automating log analysis processes within software systems [14].

However, despite their capabilities, recent LLMs, which can process large amounts of input data—referred to as the context window—are computationally expensive. This is because they require advanced GPUs with significant memory capacity to function efficiently [15]–[19].

On the other hand, language models struggle to effectively utilize information from long input contexts. Specifically, their performance tends to be best when important information appears at the beginning or end of the input, but drops significantly when the relevant information is located in the middle of long inputs [19]. Therefore choosing the best context window size is a challenge since it can be costly and directly affect the accuracy of the responses the LLM models produce.

Although, modern LLMs can automate the process of log analysis by processing and interpreting vast amounts of log data, their high computational costs and need for powerful hardware is another challenge for many companies. As a result, most companies choose to rely on external services, such as GPT models from OpenAI, to handle these tasks.

In addition to the challenges of token limits, context window, and computing costs, dealing with the privacy of log data are another big issues when using LLMs for log analysis. Logs often have sensitive details like IP addresses or user information that need to be handled carefully to follow privacy laws. Using LLMs, especially through third-party services, can risk exposing this private data [20]–[22]. Even when running LLMs on local machines, it is important to use data anonymization techniques to keep information safe. So, finding a balance between processing large amounts of logs efficiently and protecting sensitive data is a key challenge in using LLMs for log analysis. This shows the need for solutions that not only reduce token usage but also ensure privacy, which makes LLMs a secure and practical choice for organizations that manage large logs.

Based on our industrial experience, this experience report introduces a novel, adaptive approach to log file analysis using an LLM model. It focuses on solving four main challenges, listed in priority as token size limitation, data privacy with minimal loss of accuracy of LLM model responses, context window size tuning for higher accuracy, and prompt engineering for each log type and the chunks within each log. Here, *context window* size refers to the amount of text the model

can consider at once, and tuning it means adjusting this limit to capture all necessary information for accurate responses. *Prompt engineering* involves designing specific questions or instructions for the model to help it better understand and respond to each type of log.

The key contribution of this paper are as follows:

- In Section III, we propose an adaptive approach that determines whether to analyze a specific log chunk, the entire file, or a defined time window. This decision-making process is designed to address user questions effectively for root cause analysis or suspicious event detection. This technique can be applied to any dynamic data, such as logs, to focus on relevant sections based on user queries. This approach reduces the token count per call, helping to lower costs.
- We introduce and provide our experience with a novel method inspired by the principles of flame graphs [23] to help reduce the issues of token limitation and cost associated with large-scale log analysis. The flame-graph-like approach reduces input tokens by an average of 85.53% across all logs, improving performance by 75% over the baseline pipeline latency. This technique enables users to save on token costs while maintaining accurate LLM responses.
- We introduce our experience with anonymizing log data to protect sensitive information, the accuracy of our method is 138.63% better compared to general mappings. Industries and researchers can apply a similar approach to address their own data privacy concerns, reducing hallucination and improving accuracy in LLM responses.

This research has led to the development of a chatbot solution that is already in being used by developers at Ciena. A general overview of the application is shown in Figure 1.

II. BACKGROUND AND RELATED WORK

Large and complex software-intensive systems typically generate extensive log data during operation, primarily for troubleshooting and diagnosing issues [24]. However, analyzing such data at scale requires advanced techniques, as traditional methods are unable to handle the growing data volume and complexity effectively.

The recent advances in large language models (LLMs) have transformed natural language processing [25], [26]. LLMs generate text by predicting each word based on previous context using a probabilistic, autoregressive approach, making them effective for generating coherent and contextually relevant responses [27]. This capability, however, often requires adaptation techniques to handle specific tasks.

Several strategies exist for adapting language models, including fine-tuning and in-context learning. Among these, in-context learning (ICL) is a widely adopted and cost-effective approach. The core concept of ICL is learning by analogy, where a few example demonstrations are used to create a contextual prompt. These examples are often formatted using natural language templates. ICL then combines the query question with this prompt context to form an input, which

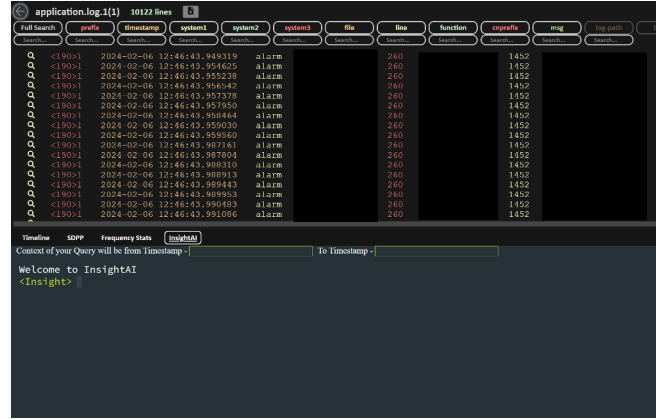


Fig. 1: An overview of InsightAI: We developed a chatbot that allows users to ask any question related to the log, and the system processes the log to find the answer. Additionally, users can select a specific time range for evaluation. The logs are displayed with a filtering feature, which lets users hide unnecessary columns and focus on the desired data. For privacy concerns regarding our internal logs, we have intentionally hidden three columns in this image.

is passed to the language model to generate a prediction [28]. ICL can significantly lower the computational expenses of adapting models for new tasks, which enables language-model-as-a-service [29] and makes it practical for large-scale, real-world applications [28].

Building on ICL, few-shot learning further enables models to handle new tasks by showing only a few examples, while Chain of Thought (CoT) prompting improves understanding by breaking down answers into step-by-step logic. Zero-shot-CoT extends this by using a simple prompt like “Let’s think step by step,”. It allows the model to handle complex tasks without any examples [30].

In this industrial experience report, we adopt zero-shot and few-shot CoT strategies in prompts when using LLMs as a service, to help generate more accurate and detailed answers across a variety of logs.

Prior research categorizes large language models (LLMs) into three primary architectures: encoder-only, decoder-only, and encoder-decoder, each suited to specific tasks. Decoder-only models like the GPT series perform well in few-shot and zero-shot tasks due to their autoregressive nature, where each word is predicted based on previous words, making them ideal for text generation and question-answering [31], [32]. We chose GPT-4o for our pipeline due to its robust performance in handling complex context-dependent log-analysis tasks.

LLMs have many strengths but also some limitations. One key limitation is the context window length, which is the maximum amount of text the model can handle at one time [26], [31], [33]. This limit is set during training, and if it is exceeded, the model can become unstable. This is a big challenge for tasks that need a larger context. Adjusting

the context window for each use case is important to keep performance high in large-scale log analysis.

To overcome the limitations of a context window, Retrieval-Augmented Generation (RAG) techniques improve model accuracy by integrating a retrieval system that finds relevant information from large datasets before generating responses. Previous work in this area, as highlighted in [34]–[38], shows the effectiveness of RAG. In our chatbot, implementing RAG enables the retrieval of relevant log chunks, which provide valuable context to the input of the model and contribute to improved accuracy.

To highlight the many uses of GPT-based chatbots, Qi et al. [39] did a survey and pointed out challenges like biases, hallucinations (creating information that sounds correct but is incorrect information), and privacy concerns. Macdonald et al. [40] also showed how LLMs can be useful in data analysis, especially when problems like context windows and privacy are handled. Since log data is very sensitive, keeping it private is still a big challenge.

To deal with privacy risks in log data, Aghili et al. [41] studied sensitive information in software logs. They found that attributes like IP addresses, timestamps, and port numbers are the most commonly considered sensitive. Their study also pointed out other important attributes, like configuration details and environmental data, which could cause privacy problems if shared. These results show the need for a good log anonymization method that not only meets privacy laws but also lowers the chance of exposing sensitive information. This is important for handling legal requirements and real-world problems faced by companies.

Below, we propose a novel industrial experience report that builds on previous work to address key challenges in using LLMs as a service for log analysis, including token limitations and the protection of sensitive information.

III. PROPOSED SOLUTION

Large companies nowadays have complex software systems made up of multiple services that communicate with each other, generating an enormous number of log files. Each log file contains a vast amount of data, and there are important correlations between these logs that are key to identifying the root cause of issues. To effectively troubleshoot problems, experts first need to understand the architecture of the application, the hierarchy of the software, and how the logs are connected. Typically, they manually extract and analyze logs to find the problem, which is both time-consuming and costly. This process is particularly challenging for new hires who may not be familiar with the system.

To help with large log analysis in our industrial setting, we developed *InsightAI*, an intelligent chatbot powered by large language models (LLMs) and based on existing LLM technology. This approach aims to address the challenges developers and maintenance teams face in identifying root causes within large volumes of log data—especially in complex environments that require knowledge of software architecture and intricate application behaviors.

With *InsightAI*, users can query log files using natural language to find issues related to software execution, configuration settings, equipment status, and other unusual events.

To support this, we built a platform that gathers diverse logs and integrated ChatGPT-4o, an advanced LLM with a 128k token limit per API call, to assist developers in automatically detecting and analyzing log-based issues. We addressed two key challenges in this setup: (1) managing the token limit per API call and (2) protecting sensitive log data during processing. The result is a fully functional web application now in use at Ciena.

Figure 2 shows our pipeline architecture, which includes five modules. We describe each module in more detail below.

A. *Decision Maker*

The decision maker processes the user query and determines the appropriate strategy for limiting the number of tokens sent to the LLM. We have four strategies to address token limitations: explicit timestamp, manual time range selection, full content evaluation, and partial content evaluation, as explained below. In general, we use intent classification to determine the type of decision we need to make for choosing the best portion of the log file. Intent classification is the process of detecting and categorizing user intents in dialogue systems, including both known (seen) and new (unseen) intents, by using methods like capsule-based approaches and zero-shot learning to handle the user intents [42].

1) *Explicit timestamp*: If the query includes a timestamp, this module extracts the center time and passes it to the active retrieval system to apply a time windowing approach, centering on the user-provided timestamp. Based on expert input at Ciena, we use a default 30-second time window for analyzing logs, which can be adjusted to make the best use of tokens for each query. This way, we send only the relevant time frame to the model, reducing token usage. The time window can also be changed depending on the type of log or the specific issue being looked at.

2) *Manual time range selection*: Our empirical evaluations indicate that developers generally have a clear understanding of the approximate time range of error occurrences, which enables a more efficient and precise examination of relevant log segments. This module provides users with the capability to manually define the desired time window for targeted analysis. A general overview is illustrated in Figure 1.

3) *Full or portion of content evaluation*: To determine whether a query requires evaluation of the entire log file or just a portion of it, we use a combination of regular expressions and an LLM-based autonomous inference. The regular expression, provided below, captures keywords indicating a need for a full-file analysis, such as “how many,” “how much,” “all,” “whole,” “total,” “summary,” and “entire.” For example, if a user asks, “How many errors occurred?” or “Provide a total summary of events,” the regular expression flags this as a query that likely requires examining the whole file.

In situations where a query targets specific sections of the log file, such as a recent time frame or a specific type of event,

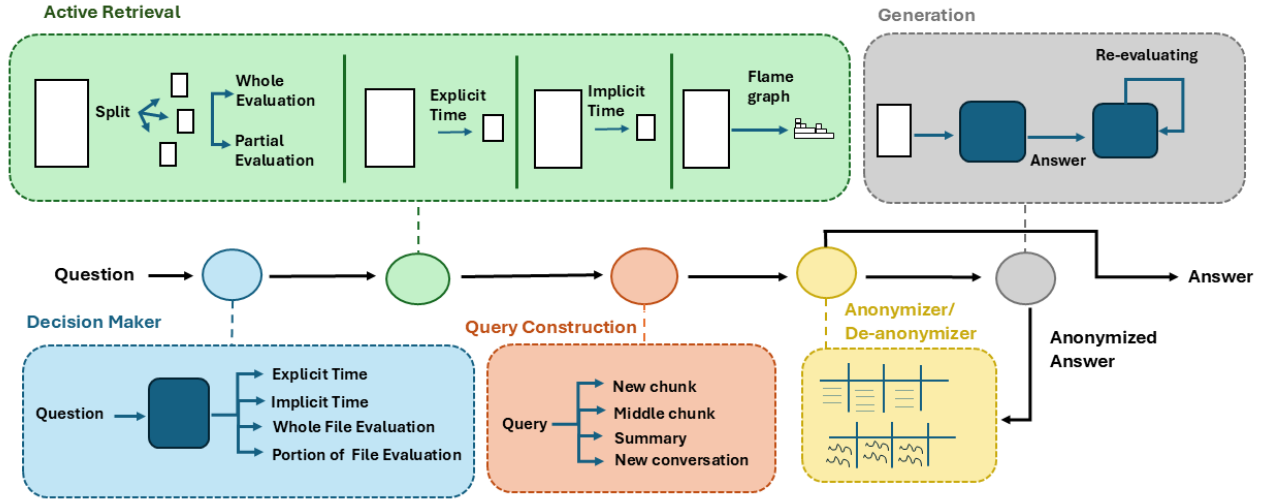


Fig. 2: InsightAI Architecture with Five Modules: Decision Maker sets extraction scope based on user queries; Anonymization sanitizes sensitive information while retaining recognizability for the model; Active Retrieval extracts data chunks or generates flame graphs; Query Construction creates queries for summaries, new chunks, middle chunks, and new conversations; and Generation Module sends queries to the LLM and re-evaluates responses.

only a portion of the file needs to be evaluated. For instance, if a user asks, “Retrieve CPU usage data for the last 10 minutes” or “What is the status of the most recent log entry?” these queries require targeted information from particular segments rather than a comprehensive analysis. This approach allows for efficient processing by narrowing down the scope to relevant sections instead of analyzing the entire log.

The regular expression and prompt template are below:

Regular expression:

```
\b(\d{1,2}:\d{2}):(\d{2})?( ?(AM|PM))?\b\d{4}-\d{2}-\d{2}\b|\b\d{2}/\d{2}/\d{4}\b|\btimestamp\b|\btime\b|\bdate\b\b
```

Prompt Template:

Analyze each query to determine the necessary scope of log file evaluation, responding with Full Evaluation for complete file analysis or Portion Evaluation for specific sections.

Examples:

Query: Identify all instances of error events throughout the log file.

Response: Full Evaluation

Query: Retrieve CPU usage data for the last 10 minutes of the log.

Response: Portion Evaluation

Query: Summarize service failures across the log file.

Response: Full Evaluation

Query: What is the status of the most recent log entry?

Response: Portion Evaluation

Template:

Query: [User Query Here]

Response:

B. Active Retrieval

This module is responsible to effectively analyze our log files based on the decision our decision-making module has

been made. Below, we will explain each decision in detail:

1) **Timestamp strategies:** The strategy involves timestamp-based extraction. If a timestamp is explicitly mentioned, the decision-making module provides a specific timestamp to the active retrieval module, which then extracts logs from a 30-second window surrounding the specified time. Should the user manually select a start and end time, the module focuses on extracting logs within this user-defined range. In the absence of specific timestamps, the log file is chunked using a recursive character text splitter (e.g., through LangChain).

2) **Portion of content evaluation:** For scenarios where only a portion of the log file needs to be analyzed, we prioritize the most relevant chunks using an embedding-based technique and calculating the cosine similarity between the query of user and the log file chunks. Based on empirical evidence and trial-and-error optimization, we found that sending the top three most relevant chunks produces more accurate results without exceeding the token limits of the model. Considering our resource limitations, we also tested several lightweight embedding models, including distilBERT-base-uncased, BERT-base-uncased, intfloat/e5-base, and TinyBERT GENERAL 4L 312D. After manually evaluating 20 ground-truth examples through domain experts, TinyBERT GENERAL 4L 312D demonstrated better accuracy compared to the other models, so we chose this model for our embedding module.

3) **Full content evaluation:** For cases where the entire log file needs to be analyzed, we have two main strategies which are our main contributions.

1- Token Count Tracking and Summarization Strategy:

For the first strategy, we keep track of the token limit. We start a new conversation for each log file the user wants to evaluate. If we hit the token limit, we summarize the previous

conversation using text summarization of GPT-4o and for its prompt we used self-prompt technique. Then we send the summary received from LLM model along with the new chunk of the log file and the query of user to a new conversation. This way, we stay within the token limits while maintaining the context of the analysis.

2- Flame-graph-like Strategy: For the second strategy, we create the **Flame-graph-like** structure to reduce the input tokens in our prompt while keeping most critical contents.

Flame graphs have become an essential tool in performance diagnostics by visually representing hierarchical patterns of system activities, making it easier to identify bottlenecks or recurring issues [43]. Inspired by this capability, we propose a method for compressing log data to optimize token usage when sending it to Large Language Models (LLMs) for analysis. The aim is to avoid exceeding token limits by using flame graph-like structures to reduce redundancy in the logs. This approach reduces token limitation keeping intact the relationships between logs and events. Instead of sending entire log files, which can be large and redundant, we group similar logs and send a flame graph representation that retains both contextual and temporal relationships. This method offers an efficient way to process log data while ensuring that relevant patterns are maintained for LLM-based analysis. An example of the flame-graph is shown in Figure 3

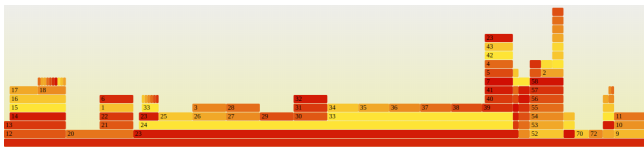


Fig. 3: An example visualization of flame graph for log sequences mapped to cosine similarity group numbers. The stacks are representative of log sequences, and the width of stacks represents the occurrences within a log file.

The flame-graph-like approach follows a multi-stage process, described below:

Step 1: Parsing and Grouping Log Messages by Similarity:

The process begins by parsing each log entry to extract the timestamp and message fields. To reduce redundancy, log messages are grouped based on their semantic similarity, using cosine similarity to measure how closely messages resemble each other. For example, consider a log file with entries like:

```
[2024-10-15 10:00:00.123] Error connecting to
database
[2024-10-15 10:00:01.456] Error connecting to
database
[2024-10-15 10:00:02.500] Failed to write to file
[2024-10-15 10:00:03.001] Error connecting to
database
[2024-10-15 10:00:03.789] Successful login for user
[2024-10-15 10:00:05.000] Error connecting to
database
```

This is achieved using cosine similarity, which compares the semantic similarity between log messages. In large log files,

certain types of messages often recur frequently. For instance, error messages like “Error connecting to database” may appear multiple times. Using cosine similarity, we group these repeated or nearly identical logs together into clusters. This reduces redundancy by ensuring that only one representative log from each cluster is transmitted to the LLM. In effect, rather than sending multiple identical logs, the system sends a single compressed version of the message, drastically reducing the number of tokens required. This approach addresses the issue of large logs by focusing on the similarity of logs and minimizing duplication.

In this file, certain messages—such as “Error connecting to database”—occur multiple times. Cosine similarity groups these repeated or nearly identical messages into clusters, resulting in groups like:

```
Group 1: Error connecting to database (occurs 4
times)
Group 2: Failed to write to file (occurs 1 time)
Group 3: Successful login for user (occurs 1 time)
```

Step2: Creating a Time-Windowed Forest of Directed Acyclic Graphs (DAGs)

The log file is used to create directed trees of logs based on their timestamps, e.g. If the next log is within the time-window of the last log, it is appended to that tree; otherwise it starts a new root stemming another tree. The process continues until we have parsed the complete log file. The algorithm can have many variations, e.g. the time window can be altered to cater to the physical problem.

```
Sequence 1: Root: Error connecting to database
(10:00:00.123)
|-- Error connecting to database (10:00:01.456)
```

```
Sequence 2: Root: Failed to write to file
(10:00:02.500)
|-- Error connecting to database (10:00:03.001)
|---- Successful login for user (10:00:03.789)
```

```
Sequence 3: Root: Error connecting to database
(10:00:05.000)
```

Step3: Merge messages

In this step, each message is replaced by its assigned group number from Step 1. For each sequence, consecutive messages with the same group number are merged within the time window, and the start and end times of each merged message are recorded. This step resembles the flame graph approach by aggregating repetitive patterns into a single, compact representation that preserves the timing and structure of events. For example, for the sequence given in Step 2, after replacing messages with group numbers and merging consecutive occurrences, the result will be:

```
Sequence 1: Group 1 (10:00:00.123 - 10:00:01.456)
```

```
Sequence 2: Root: Group 2 (10:00:02.500)
|-- Group 1 (10:00:03.001)
|---- Group 3 (10:00:03.789)
```

```
Sequence 3: Root: Group 1 (10:00:05.000)
```

By aggregating repeated log entries within each sequence and using time ranges, this step achieves a flame-graph-like

structure, compressing each sequence into a concise format. This structure effectively reduces redundancy while maintaining essential information, allowing for efficient analysis and visualization. Prompt In the initial prompt, we send grouped log messages. Within each group, we only include the dynamic part of the log message, which is the actual message content. We discard static or redundant parts of the log, such as timestamps or repetitive metadata. By doing this, we effectively reduce the number of tokens used.

Additionally, we replace the full log messages in the flame-graph-like structure with their corresponding group numbers. It represents the messages based on the group they belong to. This further reduces the tokens while keeping essential information about the sequence and timing of events. The prompt result is something like as follows:

```
Prompt = {Query query} + file_content = Below is a
list of all the similar grouped messages from
the log file (don't refer to group number in
your responses, instead refer to the specific
logs that make up the group):

Grouped log messages:
{
  "1": {
    "message": "Error connecting to database",
    "count": 4
  },
  "2": {
    "message": "Failed to write to file",
    "count": 1
  },
  "3": {
    "message": "Successful login for user",
    "count": 1
  }
}

Below is a list of events which took place within
the log file. Each number represents a log from
that respective group and the timestamps show
when the events took place (time of first and
last log in the sequence)
The following event:

a) 1 (10:00:00.123 - 10:00:01.456)
b) 2 (10:00:02.500) -> 1 (10:00:03.001) -> 3
(10:00:03.789)
c) 1 (10:00:05.000)
```

The flame graph above shows sequences of logs within specific time windows. This flame graph is different from traditional flame graphs, which come from sampled stack traces. Here is how to understand it:

- **Full Log Sequences:** Unlike regular, sample-based, flame graphs, this flame graph includes complete log sequences within each time window. There is no data loss.
- **Stacks Represent Log Sequences:** Each stack shows a sequence of logs in a time window. It follows the flow of events in that time frame.
- **Top of the Stack:** The top of each stack is the last log in the sequence. This shows where the sequence ends in the time window.

- **Stack Width and Frequency:** The width of a stack shows how many times that sequence appears in the logs. Wider stacks mean the sequence occurs more often, so common patterns stand out.
- **Forks Show Diverging Sequences:** Forks in the stack mean two log sequences meet at the same starting point (or root). This keeps important details in sub-sequences, so unique event paths remain visible.

This flame-graph-like approach compresses repeated log sequences and organizes them by time windows. It gives a simple but complete view of event patterns, which makes analysis easier and clearer.

C. Query Construction

In this Adaptive Framework, We used a self-promoting approach, inspiration from Jiao et al. [44], where the LLM model was asked to generate various prompts. Each prompt was then assessed on our dataset to identify the one that results the better responses from the LLM. We used a self-prompting strategy across all cases.

To efficiently process user queries in log analysis, we developed a tailored prompt framework with several distinct types to address query needs. The framework works as follows:

- **Time-Specific Prompts:** If the user manually selects a specific time range, we send only that selected chunk. However, when a query explicitly mentions a time frame (e.g., between “1 PM and 1:15 PM”), an Explicit Time Prompt focuses on processing logs within that window.
- **Initial Chunk Evaluation Prompt:** For the initial processing, we use a few-shot prompt as a guideline for the model. This prompt explains the log file and its types, with examples tailored to the log file name and type. This information is sent along with the user query and the first chunk for evaluation, whether the task involves analyzing the whole file or selecting the top three chunks.
- **Extended Evaluation Prompt:** For subsequent chunks, we use a different strategy depending on whether the evaluation is for the whole file or partial file. For whole-file evaluation, we ask the model to aggregate its previous responses while ensuring uniqueness and eliminating redundancies. For partial evaluation, we use a specific prompt to guide the model in analyzing the relevant chunks individually. For example, if the question is “How many services are down?”, the model analyzes the log file using parameters like module ID and status to identify shut-down services. It then combines relevant responses from previous chunks, ensuring no overlap or duplication in the final result.
- **Token Limit Management with Summarization:** When we are close to reaching the token limit for a conversation, we use two prompts to handle this condition. The first prompt summarizes all previous responses from previous conversations. The second prompt within a new conversation sends the summary of the previous conversation, user query, and new chunks of log.

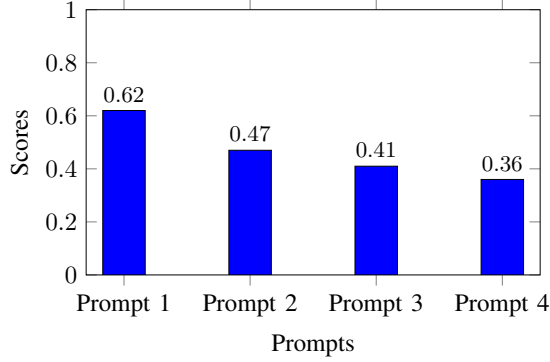


Fig. 4: F1 Scores for Different Prompts in the self-prompt strategy.

- **Self-Assessment Hallucination Mitigation:** We created self-assessment prompts to guide the model in checking its own responses. These prompts are applied to each response we receive to reduce the chance of hallucinations and increase the likelihood of relevant answers.
- **System Prompt (Instructor Prompt):** To ensure accurate and well formatted responses, we carefully select the system prompt based on some empirical studies and user feedback. Additionally, to minimize the chance of hallucinations, we incorporated chain-of-thought prompting.

As part of our query construction strategy, for each prompt, we asked the LLM model to provide suggestions. We then limited our evaluation to a portion of our logs to choose which prompts produced better responses. Below, we demonstrate some of the suggested prompts. We applied them to different log types and evaluated the responses. By calculating the F1 score in Figure 4, we selected the prompts that performed better. We repeated this process for all of our prompts.

- 1) Using the information in the following content, which explains [brief explanation], please answer this query: [User Query]. Content: [Insert Content Here].
- 2) Given the details in the provided content, which describes [brief explanation], what is the answer to this question: [User Query]? Content: [Insert Content Here].
- 3) Please find the answer to the user's question based on the content below, which includes [brief explanation]: [User Query]. Content: [Insert Content Here].
- 4) Based on the content provided, which details [brief explanation], answer the following query: [User Query]. Content: [Insert Content Here].

To avoid bias in our self-prompting and self-assessment process, such as when an analyst concentrates only on one component of the system or looks at only one part of the logs rather than all relevant logs, we used structured and standardized prompts, thereby avoiding any personal judgment at all stages. We encouraged periodic peer analyses and validated results through cross-validation of multiple log sources to ensure reliable and unbiased findings.

D. Anonymization

To ensure that no critical data is leaked outside the system, we implemented an anonymization module that sanitizes both the log file and the user query. This module anonymizes sensitive data before sending the data to the LLM. After receiving the response from the model, the system reverts the anonymized data back to its original form to preserve the privacy of sensitive data throughout the entire process. We performed a reverse mapping to restore the anonymized items to their original form before displaying them to the user.

Based on a comprehensive survey of anonymization techniques [41], we anonymized sensitive information such as IP addresses, specific names, function names, module names, and directories. Based on the experience of our experts, we found that most user queries from logs focus on message content and timestamps. Since timestamps are not considered private data and messages are embedded within functions, this information is crucial for identifying service or function correlations and understanding the sequence of events.

Since this anonymization process can make it more difficult for the model to understand and learn from the data, we conducted an evaluation in Section IV to have more accurate results from GPT-4o. For instance, random strings used in place of function names may not be easily interpreted by the model, which could impact the analysis. To solve this, we optimized our approach to make it understandable for the model and still having privacy. According to the privacy conditions of Ciena, we are unable to provide further details on the specific implementation of this module. However, we note that the techniques mentioned above work for any type of log with timestamps and messages.

E. Generation

In this module, we manage communication with the model by sending prompts generated from the query construction process. We keep track of the total token count—including both input tokens and model responses—within each conversation. When we approach the maximum token limit, we summarize the current conversation and initiate a new one to maintain context without exceeding limits. We evaluate the accuracy of the responses of the model through self-assessment strategies and integrate all responses from each chunk to form a complete and accurate answer. In case the response is inaccurate, the module will repeat asking from the active retrieval module for another chunk up to three times, which can be configured.

IV. EXPERIMENTAL EVALUATION

In this section, we evaluate the contribution of our approaches, which include optimizing the context window size, implementing a flame-graph-like approach, and ensuring effective anonymization. We collected 20 questions with expected answers as ground truth from experts at Ciena, including developers, architects, maintenance engineers, and product owners. Due to internal legal and privacy policies at Ciena, we cannot present our datasets, which include internal logs and ground truth questions with their expected answers. Our server

Anonymization	Precision (%)	Recall (%)	F1 Score (%)
RadomValue (Baseline)	86.67	23.64	37.26
FunctionNameRandomValue	100.00	80.00	88.89
FunctionName_RandomValue	100.00	76.36	86.57
FunctionName_RandomValue()	0.00	0.00	0.00
RandomValue()	100.00	27.27	42.86
functionNameRandomValue	100.00	27.27	42.86
RandomValue_FunctionName	86.67	20.00	32.47

TABLE I: Summary of anonymization results showing precision, recall, and F1 score. **FunctionNameRandomValue** demonstrates the best performance, with an F1 score improvement of approximately 138.63% over the baseline.

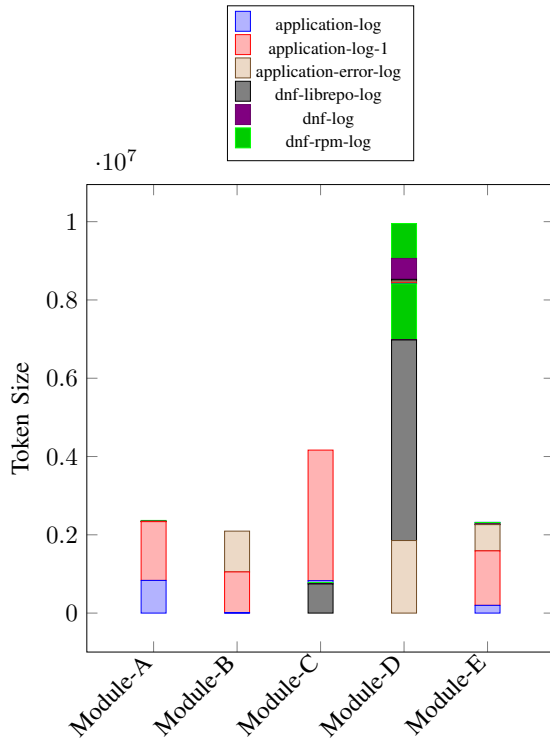


Fig. 5: Stacked bar chart illustrating the total token sizes of various log files across different Module-E modules. The x-axis represents the modules—Module-A, Module-B, Module-C, Module-D, and Module-E—while the y-axis shows the total token size for each log. Each bar segment corresponds to a specific log file as indicated in the legend: application-log, application-log-1, application-error-log, dnf-librepo-log, dnf-log, and dnf-rpm-log. This visualization highlights the distribution and magnitude of log data within each module, highlighting the total size of different internal logs.

is equipped with a 10-core Intel Xeon processor, clocked at 2.8 GHz, with a 32 KB L1d cache, a 32 KB L1i cache, a 4 MB L2 cache, and a 16 MB L3 cache, alongside 59 GB of RAM. Our system comprises five main modules, each containing multiple

services, totaling 65 log types with an average token size of 402,653 tokens, and an average log size of 1.5 GB with the range between 28 KB to 3 GB. The logs are stored in various formats, including JSON, text, CSV, and CTF, each with its own structure. However, all logs share common fields such as timestamps, line numbers, file names, function names, and message content illustrated in Figure 5. We examine how the context window size affects inference latency and accuracy, analyze the impact of total token size on the flame-graph-like approach, and assess the accuracy of our anonymization. For simplicity, we kept some parameters fixed while focusing on finding optimal values for the key ones.

	Input- output Tokens	Input Tokens	Output Tokens	Latency (Sec.)	Total Latency (Sec.)	Precision
Test 500,500	207	85	2.086	16.378	3.3%	
	225	86.5	2.103			
	244	79.5	2.155			
Test 1000,500	470	122.5	2.718	15.177	10%	
	511.5	122	2.458			
	499.5	129	2.923			
Test 1500,500	739	152.5	3.202	14.425	10%	
	725.5	106	3.59			
	724.5	106	2.956			
Test 2000,500	963	152.5	4.095	15.351	16.67%	
	963	152.5	4.190			
	963.5	152.5	3.831			
Test 2500,500	1225	127.5	2.610	12.787	13.33%	
	1223	125	3.429			
	1223	125	2.515			
Test 3000,500	1446	125	3.383	115.89	20%	
	1447.5	125	5.43			
	1449	125	3.582			
Test 3500,500	1682.5	81.5	2.472	12.96	12%	
	1668	81.5	2.472			
	1682	81.5	3.027			
Test 11000,500	2829.5	139.5	3.732	13.114	40%	
	5380	148	4.068			
	5260	153.5	3.251			
Test 20000,500	9531.5	109.5	3.524	16.55	83.33%	
	9530	109.5	4.707			
	7720.5	109.5	5.227			

TABLE II: Latency measurements per API call and chunk relevance for different input token sizes along with the precision of the responses. The table illustrates how varying input token sizes (ranging from 500 to 20,000 tokens) affects the response latency of the OpenAI GPT-4 API and the overall latency of our pipeline when processing a log file of 15,869 tokens. It also presents the relevance of the top three selected chunks to user queries.

A. The impact of the context windows size on inference accuracy and latency

In this study, due to limited computational resources, we selected lightweight embedding models, including distilBERT-base-uncased, BERT-base-uncased, intfloat/e5-base, and TinyBERT GENERAL 4L 312D. After evaluating 20 ground-truth

Name	Total Tokens	Flame-graph Tokens	Pipeline Latency (Sec.)	Flame-graph Latency (Sec.)
Log A	854 754	47 347	11.87	1.92
Log B	2179	278	8.94	0.63
Log C	1 517 318	32 772	18.44	3.67
Log D	770 823	141 728	13.65	6.55
Log E	8682	1994	2.41	0.04
Log F	1 762 090	61 854	18.13	3.91
Log G	4 764 844	177 248	27.37	10.96
Log H	1 819 004	16 074	16.75	3.91
Log I	6216	261	2.62	0.66

TABLE III: This table presents data for Logs A to I, including the total token size, the reduced token count after applying the Flame-graph-like optimization, and the execution latencies in seconds for both the Pipeline and Flame-graph-like approach. The results highlight the relationship between token size and latency. It shows that the Flame-graph-like approach reduces both the token size by approximately 77% to 99.99% and execution latency by approximately 52% to 98% across all logs compared to the general Pipeline approach.

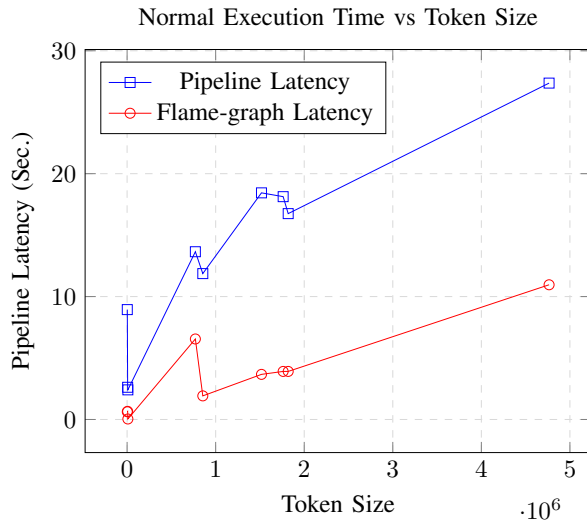


Fig. 6: Execution latency versus token size for Pipeline and Flame-graph methods. The x-axis represents the token size, and the y-axis shows the execution latency in seconds. The chart demonstrates that as the token size increases, latency rises for both methods. However, the Flame-graph-like approach consistently exhibits lower latency compared to the Pipeline method across all token sizes. The Flame-graph-like approach achieves latency reductions, with improvements ranging from 52% to over 98% compared to the Pipeline method. Thus, the Flame-graph method is more efficient, reducing latency by over 90% at smaller token sizes and by around 60% at larger token sizes. This shows the effectiveness of Flame-graph over the general Pipeline method in processing logs more quickly, regardless of token size.

examples, TinyBERT GENERAL 4L 312D emerged as the most accurate model and was therefore chosen for the embedding module. To improve chunking efficiency, we employed the Recursive Character Text Splitter from LangChain.

Since our application is in production mode and each token call has a cost, we set the output token limit to three to stay within budget. For consistency, we used two log types with an average total of 15,869 tokens in all evaluations and repeated the evaluation process 27 times to ensure reliable results.

To reduce the effect of pipeline latency, we focused not only on measuring the latency of each API call but also on evaluating the latency of the entire pipeline. This approach allowed us to see how different input token sizes affected the response time of OpenAI GPT-4o and the total latency of our basic pipeline. Table II presents detailed latency measurements for each API call, the overall pipeline latency, and the precision of the responses, allowing us to assess system performance. It shows latency per API call across token limits ranging from 500 to 20,000 tokens, including latency on individual api call and total pipeline response times. For example, tests with an 11,000-token input illustrate a trade-off: higher token limits increase latency but enhance the relevance of selected chunks. As shown in Figure 6, both per-call latency and overall pipeline latency exhibit an almost linear increase with the growth in token size. This shows the direct relationship between token size and latency. It emphasizes the importance of optimizing token usage to maintain system efficiency. To optimize this balance, we aim to find the best compromise between our budget per token call, the precision of results, and acceptable latency. In our evaluation, we found that an 11,000-token limit offered the best trade-off among these factors.

Lesson learned 1: A larger token limit led to higher latency and cost, but it also made the selected chunks more relevant. This shows a trade-off between performance and cost.

B. The impact of the log size on inference latency

We evaluated the effectiveness of a flame-graph-like approach in reducing token sizes when using LLMs as a service. For this, we applied the flame-graph-like approach to nine different logs and asked three questions for each log. Each query required a whole-file evaluation where the entire file was sent to the LLM for processing. We then measured the average latency, as shown in Table III.

Additionally, we compared the latency of this flame-graph-like approach to a general pipeline solution. The latency, defined as the time from when a user submits a query to when the result is returned, was measured for both approaches across varying token sizes. As illustrated in Figure 6, the general pipeline showed significantly higher latency as token size increased, with a peak of 27.37 seconds for 4,764,844 tokens. In contrast, our flame-graph-like solution demonstrated much more stable performance, maintaining lower latency and reaching a maximum of 10.96 seconds for the same token size.

By using our flame-graph-like approach, we achieved an average token reduction of 93.61% across all logs. This method also reduced processing latency by 77.45% compared to the general pipeline. It shows efficiency in optimizing LLM-based processing for large files.

This substantial improvement in latency shows the efficiency of flame-graph in reducing the token size and processing time. It makes a valuable optimization for LLM-based services that are sensitive to token-related costs.

Lesson learned 2: The flame-graph-like approach effectively reduces token size and processing time, achieving substantial reductions in both latency and token usage compared to standard methods. This makes it a useful way to optimize applications that need faster processing and lower token costs.

C. The impact of the anonymization and accuracy

In this module, we evaluate the effectiveness of customized anonymization compared to random mapping for private entities. Based on our expert experience, function names are the most critical data points, as they help us track events in messages, with each message originating from a specific function. Therefore, we focused on anonymizing function names specifically. Due to page limitations, we have not included the same procedure for other sensitive data, although the approach would be the same as explained here. We used the prompt below to evaluate this customized mapping and applied it across 10 of our log files.

Prompt: List all the unique function names in this log file.

As the average result shown in Table I, random mapping can sometimes reduce the ability of the module to make accurate inferences. However, our results show that adding “FunctionName” as a prefix to the randomly generated names improved module performance, providing better accuracy than other approaches.

Lesson learned 3: Applying structured prefixes and suffixes to anonymized entities can enhance model performance and accuracy in large language models by keeping important details in sensitive data. In our case, appending “FunctionName” as a prefix to anonymized function names improves model performance and accuracy in anonymized data processing.

V. LIMITATIONS

This experience report introduces a novel approach to automating log analysis that effectively reduces token usage and enhances anonymization. Despite its potential to improve system reliability and reduce maintenance costs, several limitations must be acknowledged:

- In this project, we faced several resource limitations, particularly regarding computational resources like GPU and memory. As a result, we were constrained to use

lightweight embedding vectors, which led to reduced accuracy in retrieving the most relevant chunks.

- Due to budget limitations associated with token calls, we limited our parameter tuning process to the top three chunks. We recommend that future researchers consider adjusting these limitations in their own applied projects to achieve more accurate results.
- Due to the internal policies of the company and privacy concerns, GPT-4o is the designated model, limiting access to other models. However, in the future, we may be able to evaluate additional models for comparison.

Future researchers are highly encouraged to perform comparisons between models, including open-source LLMs, and to apply the proposed pipeline and flame-graph-like approach to various datasets. Additionally, while each dataset has its own unique privacy considerations, the general anonymization approach outlined here can be adapted to meet specific privacy needs.

VI. CONCLUSION

Many companies face challenges when processing and analyzing large amounts of log data. Software logs are dynamic and constantly changing, making it difficult to evaluate them manually. For anomaly detection or any other investigation, large language models (LLMs) can help automate this process. However, due to token limits and high computational demands, handling very large logs in-house remains challenging. Given these constraints, it is often more practical for companies to rely on external LLM models. However, using LLMs for log analysis can be costly, and data privacy is a concern, especially when relying on LLM as a service.

In this industrial experience, we proposed a flame-graph-like approach to reduce the input tokens required for log analysis. Our method achieved an average reduction of 93.61% in input tokens across all tested logs, effectively overcoming token limits. This approach also enhanced computational efficiency, reducing processing time by 77.45% compared to the basic pipeline.

Our dynamic pipeline adapts to user queries, whether they need insights from a specific log segment, the entire log file, or a particular time frame. We also proposed a solution to anonymize sensitive data in logs, which improved response accuracy and reduced hallucinations compared to general mapping methods.

Each stage of our pipeline provided valuable insights that we believe will be useful for future researchers and industry practitioners.

ACKNOWLEDGMENT

This work is supported by Ciena. We would also like to thank the Natural Sciences and Engineering Research Council of Canada (NSERC), Ericsson, and EfficiOS for supporting this research. We also thank Prince Kapoor and our colleagues at Ciena for their collaboration in this project.

REFERENCES

- [1] J. Xu, Z. Cui, Y. Zhao, X. Zhang, S. He, P. He, L. Li, Y. Kang, Q. Lin, Y. Dang *et al.*, “Unilog: Automatic logging via llm and in-context learning,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.
- [2] B. Chen and Z. M. Jiang, “A survey of software log instrumentation,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–34, 2021.
- [3] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM computing surveys (CSUR)*, vol. 54, no. 6, pp. 1–37, 2021.
- [4] F. F. Daneshgar, M. Dagenais, M. Ekhlesi *et al.*, “Dtracomp: Comparing distributed execution traces for understanding intermittent latency sources,” *Authorea*, October 18 2023.
- [5] M. Du, F. Li, G. Zheng, and V. Srikumar, “Deeplog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 1285–1298.
- [6] S. He, J. Zhu, P. He, and M. R. Lyu, “Experience report: System log analysis for anomaly detection,” in *2016 IEEE 27th international symposium on software reliability engineering (ISSRE)*. IEEE, 2016, pp. 207–218.
- [7] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, “Mining invariants from console logs for system problem detection,” in *2010 USENIX Annual Technical Conference (USENIX ATC 10)*, 2010.
- [8] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li *et al.*, “Robust log-based anomaly detection on unstable log data,” in *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 807–817.
- [9] W. E. Wong, V. Debroy, R. Golden, X. Xu, and B. Thuraisingham, “Effective software fault localization using an rbf neural network,” *IEEE Transactions on Reliability*, vol. 61, no. 1, pp. 149–169, 2011.
- [10] D.-Q. Zou, H. Qin, and H. Jin, “Uilog: Improving log-based fault diagnosis by log analysis,” *Journal of computer science and technology*, vol. 31, no. 5, pp. 1038–1052, 2016.
- [11] S. Lu, B. Rao, X. Wei, B. Tak, L. Wang, and L. Wang, “Log-based abnormal task detection and root cause analysis for spark,” in *2017 IEEE International Conference on Web Services (ICWS)*. IEEE, 2017, pp. 389–396.
- [12] N. Gurumdimma, A. Jhumka, M. Liakata, E. Chuah, and J. Browne, “Crude: Combining resource usage data and error logs for accurate error detection in large-scale distributed systems,” in *2016 IEEE 35th Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2016, pp. 51–60.
- [13] S. Petrescu, F. Den Hengst, A. Uta, and J. S. Rellermeier, “Log parsing evaluation in the era of modern software systems,” in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2023, pp. 379–390.
- [14] E. Karlsen, X. Luo, N. Zincir-Heywood, and M. Heywood, “Benchmarking large language models for log analysis, security, and interpretation,” *Journal of Network and Systems Management*, vol. 32, no. 3, p. 59, 2024.
- [15] Z. Dai, “Transformer-xl: Attentive language models beyond a fixed-length context,” *arXiv preprint arXiv:1901.02860*, 2019.
- [16] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [17] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré, “Hyena hierarchy: Towards larger convolutional language models,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 28 043–28 078.
- [18] O. Rubin and J. Berant, “Long-range language modeling with self-retrieval,” *arXiv preprint arXiv:2306.13421*, 2023.
- [19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [20] V. Patil, P. Hase, and M. Bansal, “Can sensitive information be deleted from llms? objectives for defending against extraction attacks,” *arXiv preprint arXiv:2309.17410*, 2023.
- [21] J. G. Wang, J. Wang, M. Li, and S. Neel, “Pandora’s white-box: Increased training data leakage in open llms,” *arXiv preprint arXiv:2402.17012*, 2024.
- [22] B. Yan, K. Li, M. Xu, Y. Dong, Y. Zhang, Z. Ren, and X. Cheng, “On protecting the data privacy of large language models (llms): A survey,” *arXiv preprint arXiv:2403.05156*, 2024.
- [23] B. Gregg, “The flame graph: This visualization of software execution is a new necessity for performance profiling and debugging,” *Queue*, vol. 14, no. 2, p. 91–110, Mar. 2016. [Online]. Available: <https://doi.org/10.1145/2927299.2927301>
- [24] V.-H. Le and H. Zhang, “Log parsing: How far can chatgpt go?” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 1699–1704.
- [25] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill *et al.*, “On the opportunities and risks of foundation models,” *arXiv preprint arXiv:2108.07258*, 2021.
- [26] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [27] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [28] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, T. Liu *et al.*, “A survey on in-context learning,” *arXiv preprint arXiv:2301.00234*, 2022.
- [29] T. Sun, Y. Shao, H. Qian, X. Huang, and X. Qiu, “Black-box tuning for language-model-as-a-service,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 20 841–20 855.
- [30] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022.
- [31] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, and X. Hu, “Harnessing the power of llms in practice: A survey on chatgpt and beyond,” *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 6, pp. 1–32, 2024.
- [32] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [33] H. Jin, X. Han, J. Yang, Z. Jiang, Z. Liu, C.-Y. Chang, H. Chen, and X. Hu, “Llm maybe longlm: Self-extend llm context window without tuning,” *arXiv preprint arXiv:2401.01325*, 2024.
- [34] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “Retrieval augmented language model pre-training,” in *International conference on machine learning*. PMLR, 2020, pp. 3929–3938.
- [35] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [36] Y. Lyu, Z. Li, S. Niu, F. Xiong, B. Tang, W. Wang, H. Wu, H. Liu, T. Xu, and E. Chen, “Crud-rag: A comprehensive chinese benchmark for retrieval-augmented generation of large language models,” *ACM Transactions on Information Systems*, 2024.
- [37] R. Nogueira and K. Cho, “Passage re-ranking with bert,” *arXiv preprint arXiv:1901.04085*, 2019.
- [38] M. Besta, A. Kubicek, R. Niggli, R. Gerstenberger, L. Weitzendorf, M. Chi, P. Iff, J. Gajda, P. Nyczzyk, J. Müller *et al.*, “Multi-head rag: Solving multi-aspect problems with llms,” *arXiv preprint arXiv:2406.05085*, 2024.
- [39] S. K. Dam, C. S. Hong, Y. Qiao, and C. Zhang, “A complete survey on llm-based ai chatbots,” *arXiv preprint arXiv:2406.16937*, 2024.
- [40] C. Macdonald, D. Adeloey, A. Sheikh, and I. Rudan, “Can chatgpt draft a research article? an example of population-level vaccine effectiveness analysis,” *Journal of global health*, vol. 13, 2023.
- [41] R. Aghili and other authors, “An empirical study of sensitive information in logs,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.11313>
- [42] S. L. Devi and R. R. Pattabhi, “Intent detection and zero-shot intent classification for chatbots,” in *Proceedings of the 20th International Conference on Natural Language Processing (ICON)*, 2023, pp. 636–640.
- [43] B. Gregg, “The flame graph,” *Communications of the ACM*, vol. 59, no. 6, pp. 48–57, 2016.
- [44] W. Jiao, W. Wang, J.-t. Huang, X. Wang, S. Shi, and Z. Tu, “Is chatgpt a good translator? yes with gpt-4 as the engine,” *arXiv preprint arXiv:2301.08745*, 2023.